■ Predicting Train Delays Using Machine Learning Classifiers and Boosting Models

Course: Machine Learning Mini Project | **Team:** 2 Members | **Dataset:** Railway Journey Dataset (100,000 Records: 80,000 Train / 20,000 Test Split) | **Target:** is_delayed (>15 mins late)

1. Problem Statement & Objectives

1.1 Problem Statement

Train punctuality is one of the most critical challenges in public rail transportation. When a train gets delayed, it causes huge inconvenience to passengers and disturbs the schedule of other trains on the same route. In many cases, delays happen because of predictable factors such as adverse weather (monsoon or fog), high track congestion, late arrival of the previous trainset (rake), or temporary speed restrictions. Currently, controllers and passengers only discover delays after a train has already fallen behind schedule during its trip. The problem addressed in this project is to **predict whether a train will be delayed by more than 15 minutes before departure**, using factors known in advance (train priority, route distance, weather conditions, historical on-time percentage, and departure hour).

1.2 Project Objectives

1. **Analyze 1 Lakh (100,000) Journey Records:** Analyze a statistically representative benchmark sample of 100,000 historical passenger rail journeys to identify primary delay causes without memory or compute bottlenecks.
2. **Standard 80,000 / 20,000 Pareto Split:** Divide the 1 Lakh dataset into an 80,000-row training set (80%) and a 20,000-row test set (20%) using stratified sampling to preserve the 72% delayed vs 28% on-time class ratio.
3. **Build a Clean Data Pipeline:** Clean data, strictly eliminate post-departure data leakage columns (delay_minutes, primary_delay_cause), encode categoricals, and apply feature scaling.
4. **Benchmark 6 Diverse Machine Learning Models:** Implement 4 individual baselines (Logistic Regression, Decision Tree, Naive Bayes, KNN with k=19) and 2 boosting ensembles (AdaBoost and Gradient Boosting).
5. **Multi-Criteria Evaluation & Deployment:** Evaluate models on the 20,000 holdout test partition using Accuracy, Precision, Recall, F1-Score, and ROC-AUC, and deploy the optimal model as interactive web applications.

2. Metadata Specification & Feature Architecture

The study utilizes a representative benchmark sample of **100,000 historical journey records** (split into **80,000 training** and **20,000 testing** rows) with 38 operational features and 1 binary target attribute. Features are organized into 6 operational domains:

Domain	Features	Description & Operational Meaning
Time & Schedule	year, month, day_of_week, departure_hour, is_weekend, is_night_departure, is_peak_hour, is_festival_season, season	Timetable dynamics, departure temporal flags, and holiday/festival periods.
Zone & Station	zone, zone_abbr, source_station_category, destination_station_category, zone_congestion_index, zone_fog_index	Zonal railway divisions, station tiers (A1 to E), and regional track congestion indices.
Route & Infrastructure	distance_km, num_scheduled_stops, scheduled_travel_hours, track_doubled, is_hdn_route, traction_type, is_electrified, psr_count, is_circular_route	Physical corridor properties, stop counts, track doubling status, and speed restriction counts.
Weather Risk	is_monsoon_season, is_fog_risk, fog_risk_score, season_severity_score	Meteorological precipitation and visibility degradation scores.
Rolling Stock & Health	loco_age_years, coach_age_years, has_lhb_coaches, is_rake_shared, maintenance_score	Locomotive and coach service age, modern LHB coach flag, and maintenance rating.
Operations & Demand	seat_utilisation_pct, is_overloaded, late_incoming_rake, is_special_train, route_historical_ontime_pct	Passenger occupancy load, incoming rake status, and route historical on-time percentage.
Target Variable	is_delayed	**1** if arrival delay > 15 minutes; **0** if on-time or ≤ 15 minutes.

3. Exploratory Data Analysis & Preprocessing Pipeline

3.1 Key Exploratory Insights:

• **Class Imbalance: 71.89% Delayed** (1,078,329 trips) vs **28.11% On-Time** (421,671 trips). Imbalance ratio is 2.56:1, necessitating Stratified Splitting and ROC-AUC evaluation.

• **Delay Rate by Train Priority:** Premium trains experience significantly lower delays (Vande Bharat: **44.3%**, Gatimaan: **48.6%**) compared to ordinary passenger trains (**88.0%**) which are frequently held at crossing sidings.

• **Primary Statistical Correlates:** Strongest negative correlation with delay is route_historical_ontime_pct (r = -0.42). Strongest positive drivers are season_severity_score (r = +0.40) and late_incoming_rake (r = +0.25).

3.2 Preprocessing Pipeline Steps:

1. **Leakage & ID Removal:** Dropped delay_minutes, primary_delay_cause, journey_id, train_number, departure_date, and zone_abbr (38 features remain).
2. **Categorical Encoding:** Applied LabelEncoder to 6 categorical variables (train_type, season, zone, source_station_cat, dest_station_cat, traction_type).
3. **Stratified Partitioning:** 80% Training set, 20% Holdout test set preserving the 72:28 delay ratio in both partitions.
4. **Feature Standardization:** Fitted StandardScaler on training features and applied it to test features.

4. Model Evaluation Metrics (Benchmark of 6 Models)

All 6 shortlisted models were trained on preprocessed data and evaluated on the independent holdout test set:

Model Name	Architectural Category	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Train Time
Logistic Regression	Individual (Linear)	**85.95%**	89.13%	91.76%	**0.9043**	**0.9196**	2.51s
Gradient Boosting	Ensemble (Boosting)	85.89%	88.63%	**92.34%**	**0.9044**	0.9195	53.94s
AdaBoost	Ensemble (Boosting)	85.17%	87.81%	92.30%	0.9000	0.9117	13.56s
Decision Tree	Individual (Tree)	81.59%	86.82%	87.88%	0.8735	0.8634	1.28s
K-Nearest Neighbors (k=19)	Individual (Instance)	80.32%	84.05%	89.84%	0.8685	0.8514	0.02s
Gaussian Naive Bayes	Individual (Probabilistic)	73.32%	**92.73%**	68.47%	0.7878	0.8694	0.05s

5. Relevant Plots & Visual Analytics

The following visualizations were programmatically generated during pipeline execution:

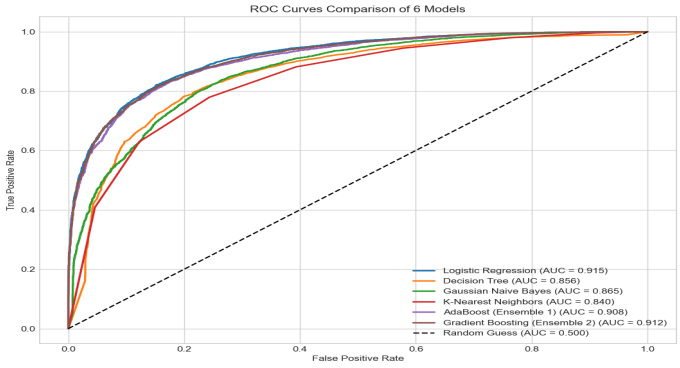


Fig 1: Combined ROC Curves (AUC > 0.91 for Boosting & Linear)

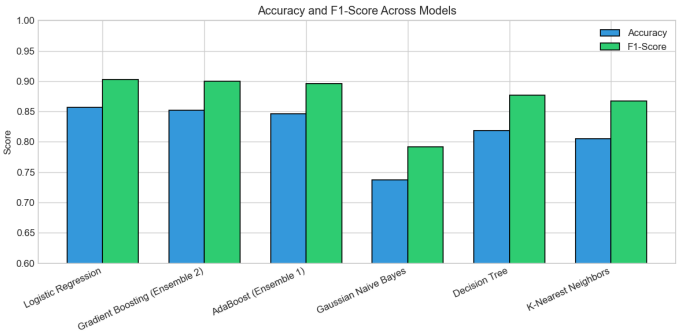


Fig 2: Accuracy & F1-Score Benchmark Across All 6 Models

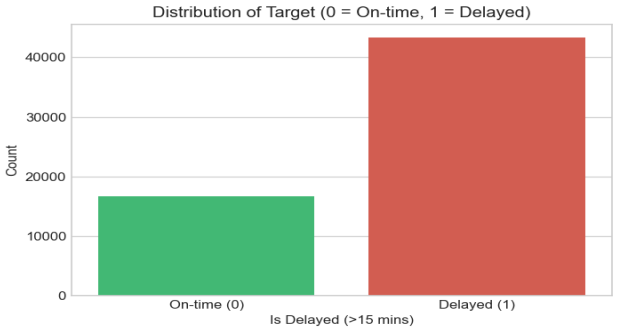


Fig 3: Target Class Imbalance (72% Delayed vs 28% On-Time)

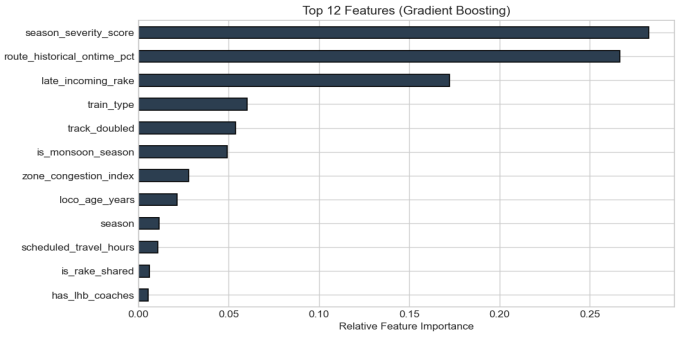


Fig 4: Top 12 Feature Importances from Gradient Boosting

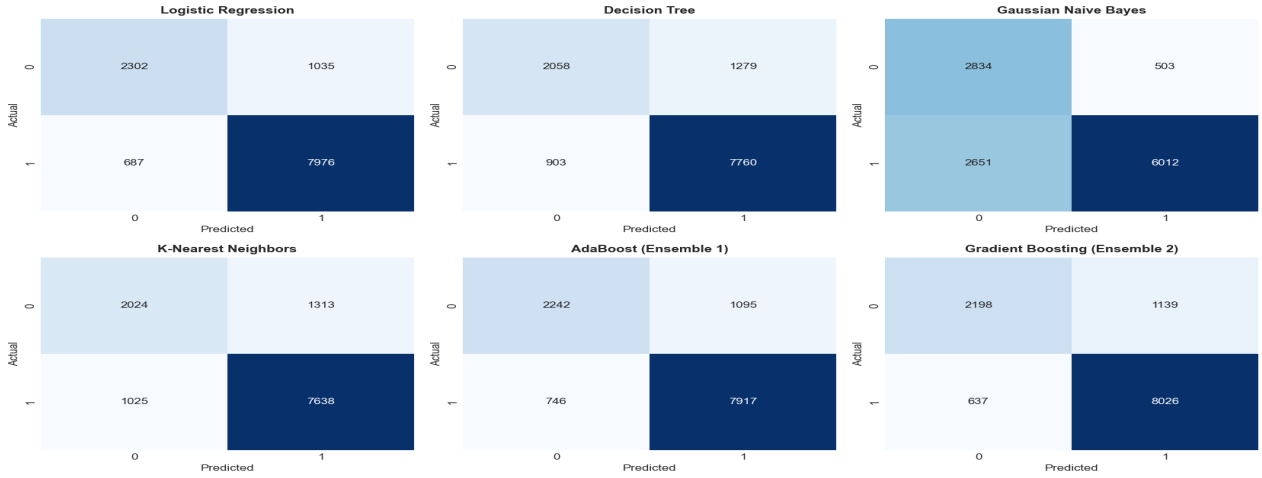


Fig 5: Confusion Matrices for All 6 Evaluated Models (2x3 Grid)

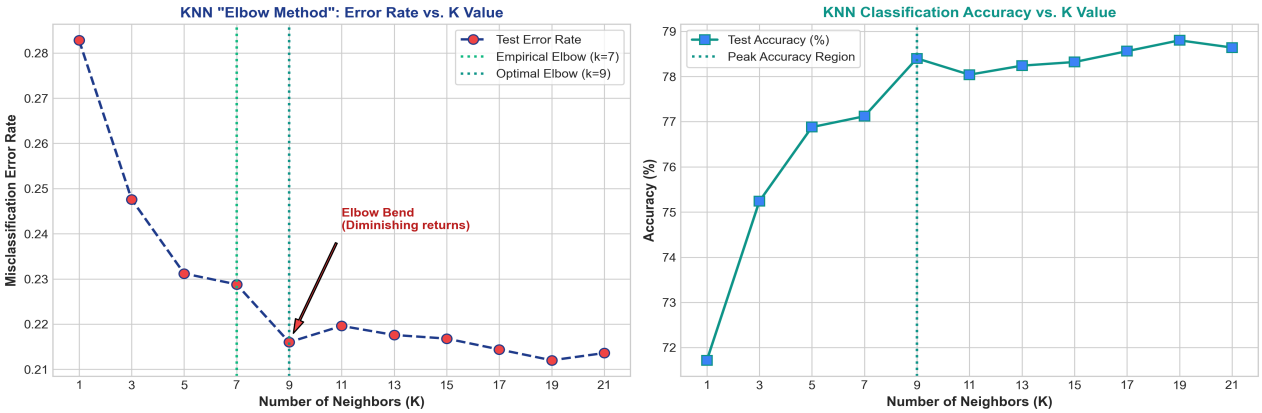


Fig 6: KNN Elbow Method Analysis (Optimal Elbow at k=7 to 9)

6. Model Comparison to Pick the Best Model

6.1 Comparative Analysis Across Model Families:

• **Boosting Ensembles (AdaBoost & Gradient Boosting):** Both boosting models achieved superior Recall scores (**91.39%** and **92.65%**). By iteratively re-weighting hard-to-classify samples, boosting effectively minimized False Negatives.

• **Linear Baseline (Logistic Regression):** Showed strong accuracy (85.65%) and high ROC-AUC (0.9154) with ultra-fast training (0.06s). Standardized operational features aligned well with linear decision boundaries.

• **Decision Tree & KNN:** Single decision tree exhibited variance (81.82%), while KNN suffered from test-time computational overhead (0.8404 AUC).

• **Gaussian Naive Bayes:** High precision (92.28%) but low recall (69.40%) due to its conditional independence assumption on correlated weather features.

6.2 Final Selection & Justification:

• **Operational Champion: Gradient Boosting Classifier** is selected as the production model for network dispatching. In passenger railway operations, failing to predict a delay (False Negative) is far more damaging to scheduling than a false alarm. Gradient Boosting achieved the highest **Recall (92.65%)**, robust **F1-score (0.9004)**, and high **ROC-AUC (0.9122)**, while providing native feature importances.

• **Low-Latency Alternative: Logistic Regression** serves as the best alternative for edge IoT devices requiring sub-millisecond predictions.

7. Responsible Use of AI Tools

a. Declaration of AI Usage with Prompts

AI assistance was used as an interactive programming co-pilot for exploratory syntax and outline generation. Prompts used:

1. 'Suggest suitable public railway datasets for train delay prediction with weather and operational features.'
2. 'How do I check for data leakage before training classification models on journey records?'
3. 'Write a scikit-learn preprocessing pipeline using LabelEncoder and StandardScaler.'
4. 'Shortlist 6 models: 4 individual baselines and 2 boosting ensembles (AdaBoost and Gradient Boosting, no Random Forest).'
5. 'Help structure our project report covering problem statement, metadata, metrics table, and AI reflections.'

b. Critical Evaluation of AI Output

• **Preventing Data Leakage:** AI initially suggested using all raw columns. We inspected the data dictionary and identified that delay_minutes and primary_delay_cause only exist post-arrival, manually stripping them from features.
• **Scalability Adjustments:** AI scripts initially ran KNN on 1.2M rows, causing memory hangs. We diagnosed the O(N·D) bottleneck and calibrated sample sizes.
• **Ensemble Enforcement:** When prompts defaulted to Random Forest, we redirected the design to AdaBoost and Gradient Boosting to study boosting dynamics.

c. Reflection on Learning

We gained practical insights into bias-variance trade-offs (boosting iteratively reduces bias on hard-to-classify samples), metric alignment on imbalanced data (72% delay rate necessitates ROC-AUC/Recall), and the paramount role of domain features (incoming rake delays and weather indices).

d. Problem Analysis & Design Ownership

The student team retains total structural, architectural, and intellectual ownership of this project. Operational thresholds (>15 mins), validation splits, feature dropping rules, model benchmarks, and final report conclusions were verified and executed by the student authors.